Not Running

For the best experience, choose the region closest to you. Then, start the lab to begin.

MARK UNFINISHED

Select the region closest to you

US-East

Keyboard layout: US English (US)

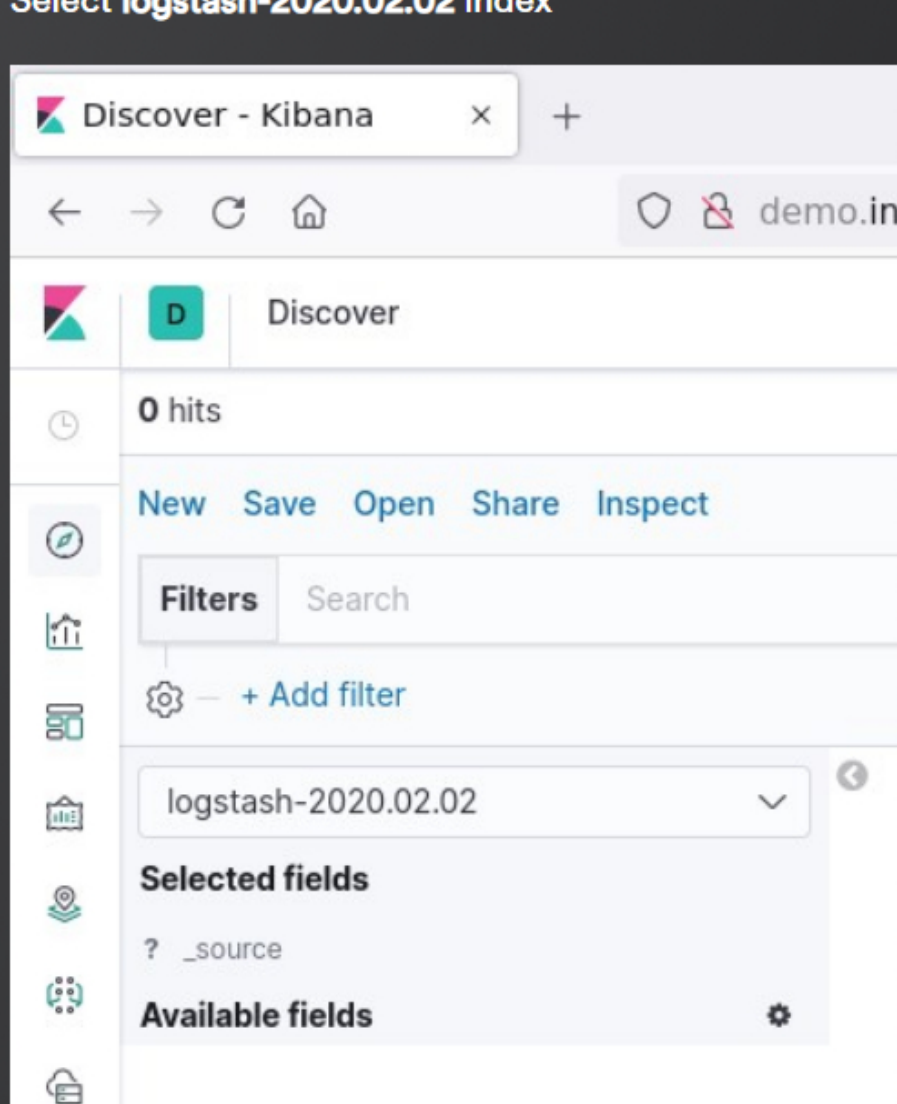
Start lab

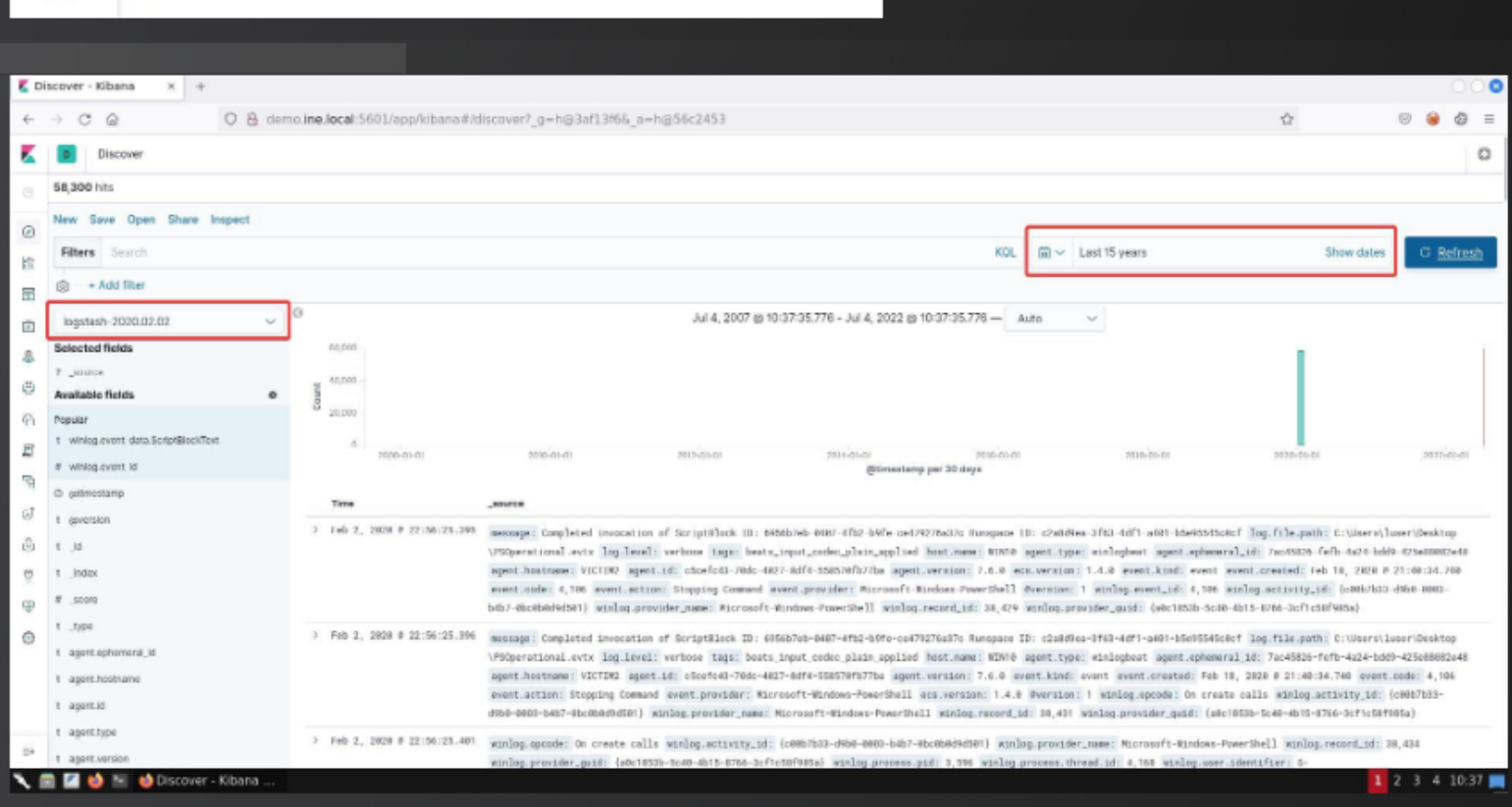
SOLUTIONS

Below, you can find solutions for each task. Remember though, that you can follow your own strategy, which may be different from the one explained in the following lab.

Kali Machine

Select **logstash-2020.02.02** index


Discover - Kibana
demo.in
Discover
0 hits
New Save Open Share Inspect
Filters Search
+ Add filter
logstash-2020.02.02
Selected fields
? _source
Available fields

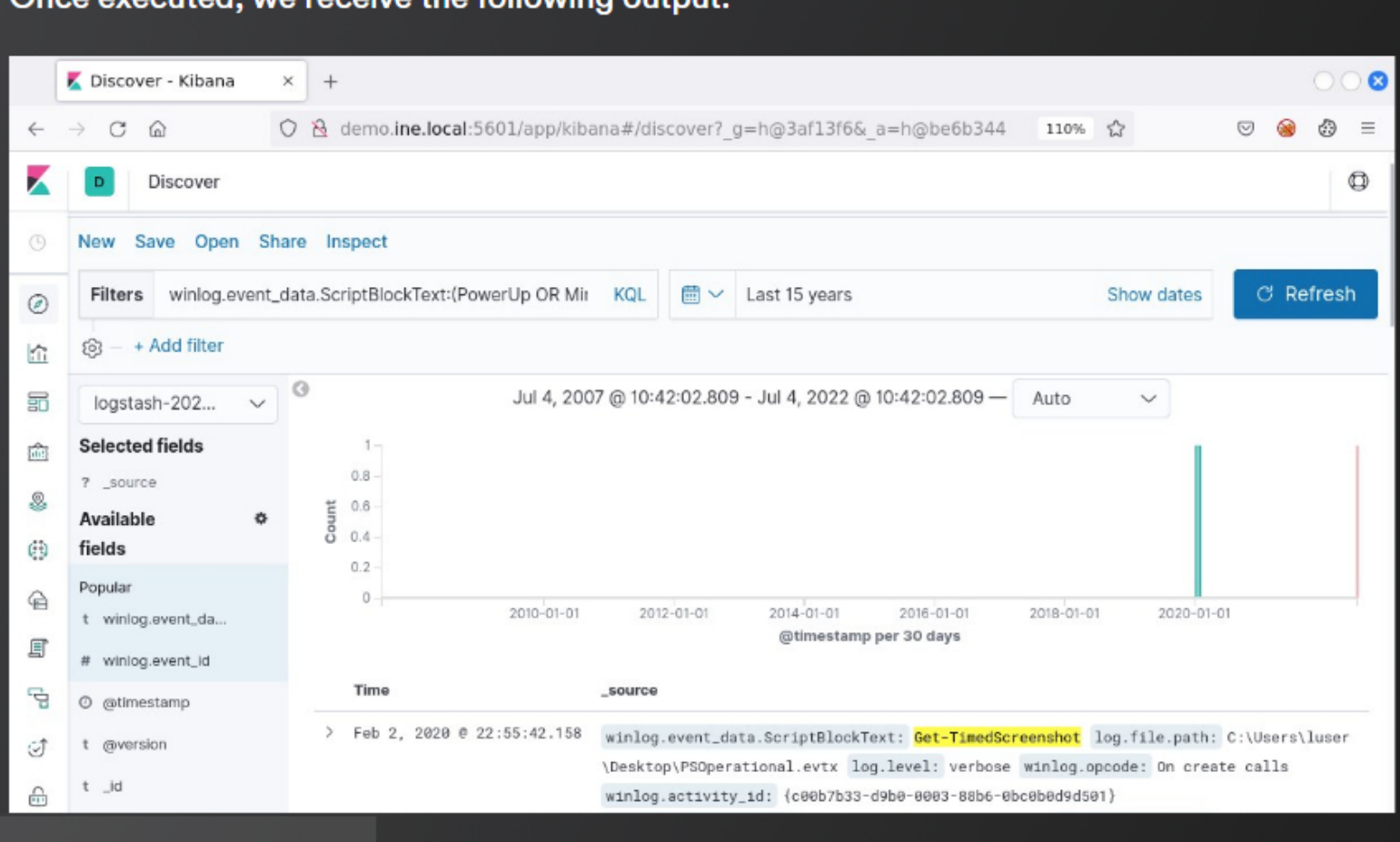


Task 1. Perform a hunt for well-known PowerShell Offensive Frameworks and commands

The information we are interested in is contained in the ScriptBlockText field of event ID 4104. Therefore, to begin with, we set a filter in ELK for event ID 4104. We then proceed into constructing the following query after performing a search on popular PowerShell Frameworks/Commands:

```
winlog.event_data.ScriptBlockText:(PowerUp OR Mimikatz OR NinjaCopy OR Get-ModifiablePath OR
```

Once executed, we receive the following output:


Discover - Kibana
demo.ine.local:5601/app/kibana#/discover?_g=h@3af13f6&_a=h@be6b344
Discover
New Save Open Share Inspect
Filters winlog.event_data.ScriptBlockText:(PowerUp OR Mi KQL Last 15 years Show dates Refresh
Jul 4, 2007 @ 10:42:02.809 - Jul 4, 2022 @ 10:42:02.809 — Auto
@timestamp per 30 days
Time _source

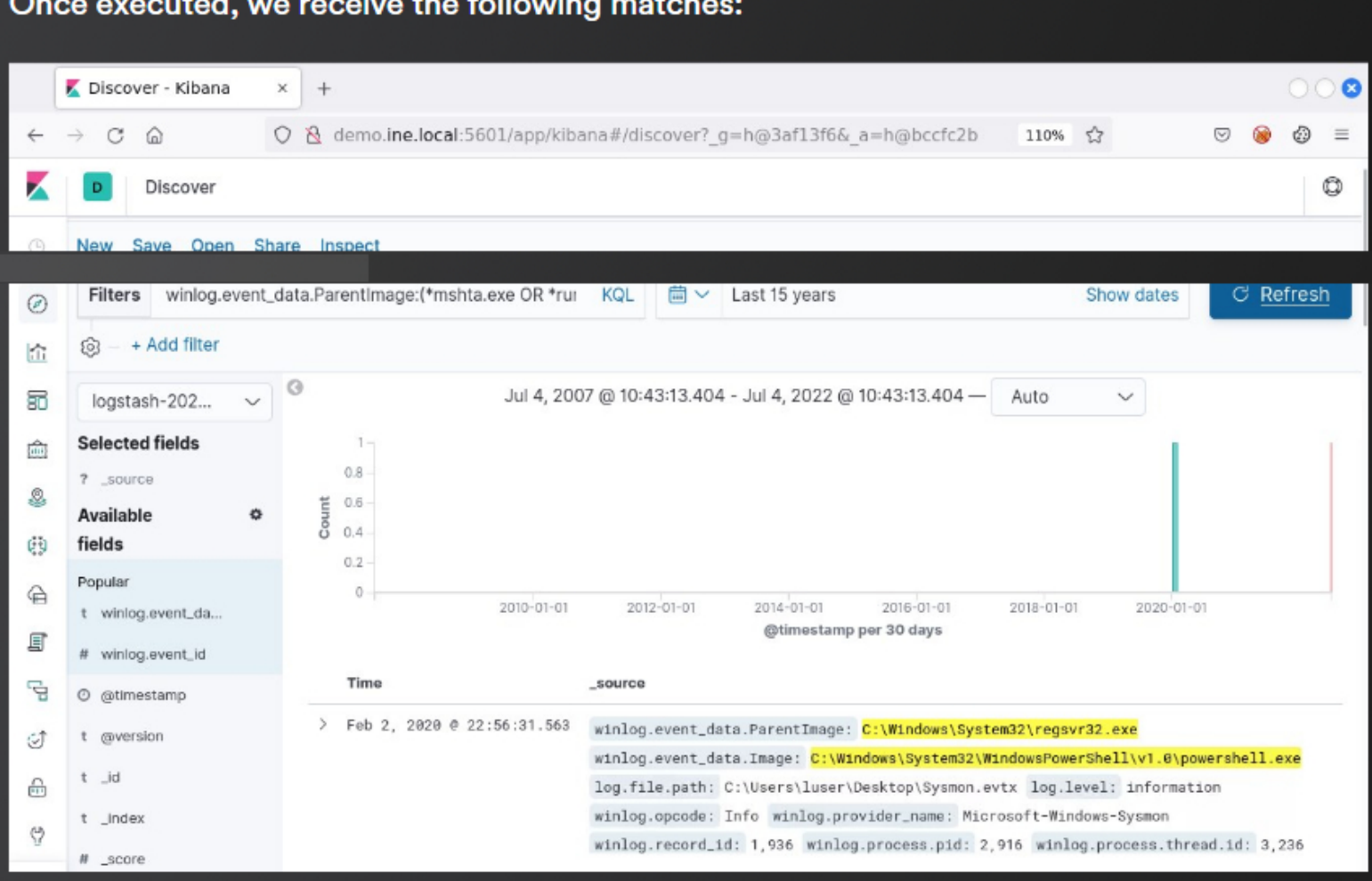
```
Feb 2, 2020 @ 22:55:42.158  winlog.event_data.ScriptBlockText: Get-TimedScreenshot log.file.path: C:\Users\user
\Desktop\PSOperational.evtx log.level: verbose winlog.opcode: On create calls
```

Task 2. Perform a hunt for suspicious parent process spawning PowerShell

For this task, we'll look into Sysmon's Process creation events, event id 1 so we start by filtering out for those. Then we proceed with our research for identifying suspicious parent processes. To construct the following query with the data researched, we should remember that it will only be valid if it spawns a process "powershell.exe" (let's omit the fact that a renamed PowerShell could be spawned for now):

```
winlog.event_data.ParentImage:(*mshta.exe OR *rundll32.exe OR *regsvr32.exe OR *services.exe
```

Once executed, we receive the following matches:


Discover - Kibana
demo.ine.local:5601/app/kibana#/discover?_g=h@3af13f6&_a=h@bccfc2b
Discover
Filters winlog.event_data.ParentImage:(*mshta.exe OR *ru KQL Last 15 years Show dates Refresh
Jul 4, 2007 @ 10:43:13.404 - Jul 4, 2022 @ 10:43:13.404 — Auto
@timestamp per 30 days
Time _source

```
Feb 2, 2020 @ 22:56:31.583  winlog.event_data.ParentImage: C:\Windows\System32\regsvr32.exe
winlog.event_data.Image: C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
log.file.path: C:\Users\user\Desktop\Sysmon.evtx log.level: information
winlog.opcode: Info winlog.provider_name: Microsoft-Windows-Sysmon
```

If you expand the first match and look at the parent process's command line argument, you'll notice the following which indeed looks suspicious:

```
t  winlog.event_data.ParentCommandLine  regsvr32.exe  /s /u /i:shotter.sct scrobj.dll
```

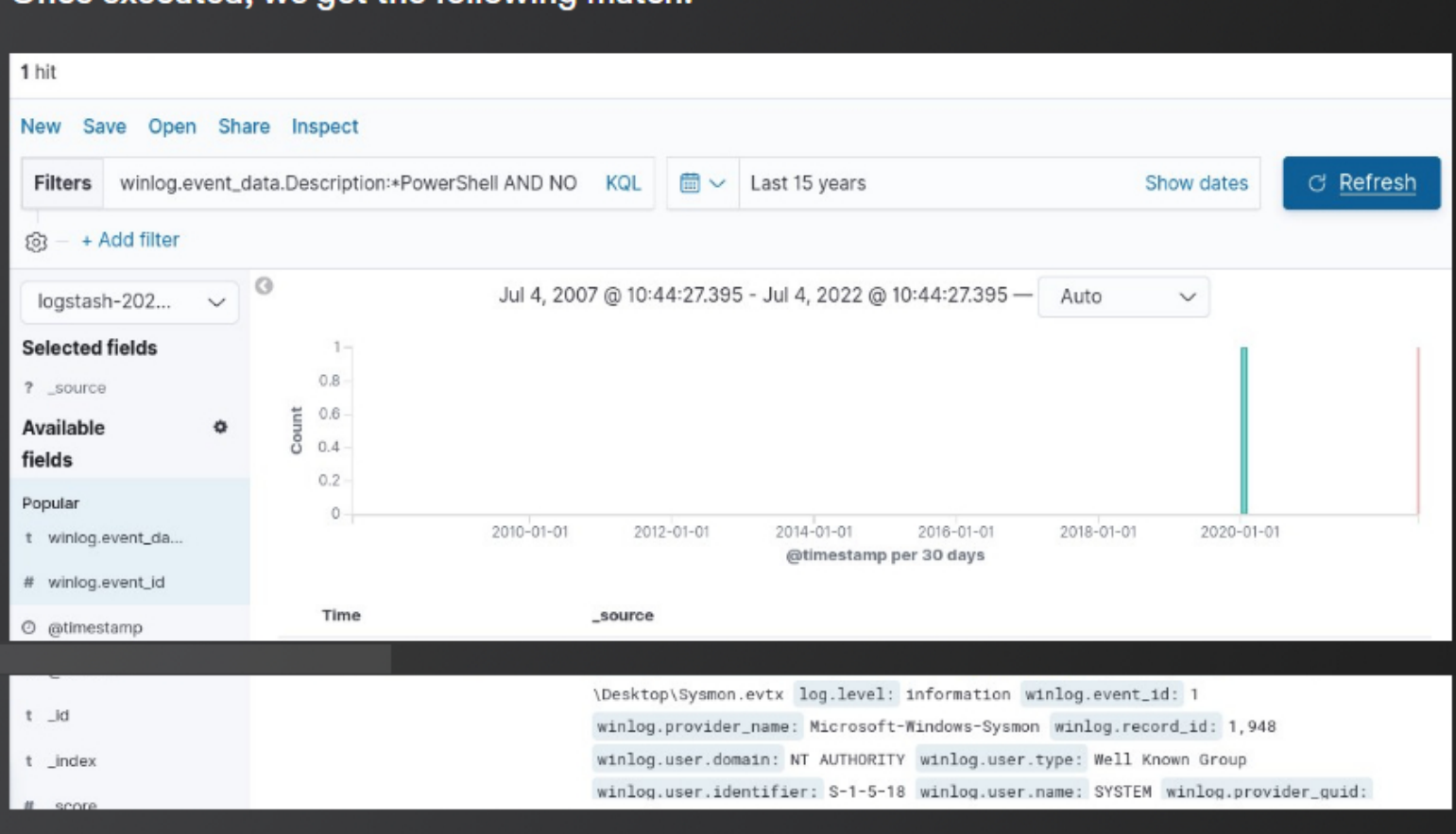
Regsvr32 using a well-known command execution to spawn PowerShell.

Task 3. Perform a hunt for renamed PowerShell.exe

Our detection is based on the fact that the PowerShell executable preserves "PowerShell" in its description, regardless of the name. So, we look into Sysmon again for any created process (event id 1) with description containing "PowerShell" that is not powershell.exe or powershell_ise.exe. We construct the following query:

```
winlog.event_data.Description:*PowerShell AND NOT (winlog.event_data.Image:*powershell.exe OR
```

Once executed, we get the following match:


1 hit
New Save Open Share Inspect
Filters winlog.event_data.Description:*PowerShell AND NO KQL Last 15 years Show dates Refresh
Jul 4, 2007 @ 10:44:27.395 - Jul 4, 2022 @ 10:44:27.395 — Auto
@timestamp per 30 days
Time _source

```
\Desktop\Sysmon.evtx log.level: information winlog.event_id: 1
winlog.provider_name: Microsoft-Windows-Sysmon winlog.record_id: 1,940
winlog.user.domain: NT AUTHORITY winlog.user.type: Well Known Group
winlog.user.identifier: S-1-5-18 winlog.user.name: SYSTEM
```

The image above reveals that the program executed is "C:\ProgramData\Windows.exe"

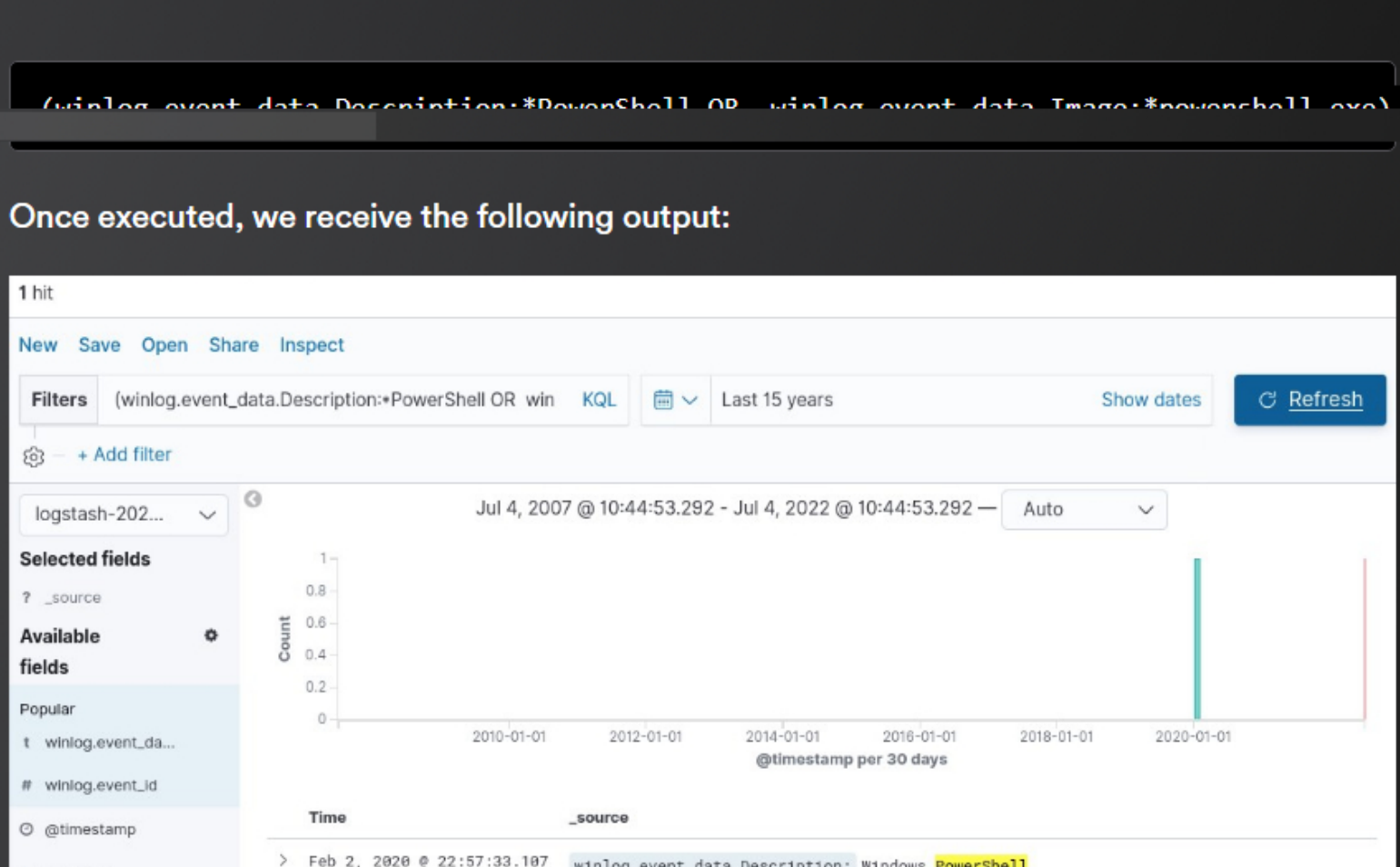
Another approach would be to look for EventID 400, where the HostName is ConsoleHost but the HostApplication is not powershell.exe.

Task 4. Perform a hunt for base64-encoded PowerShell commands

For this part, we'll use Sysmon event id 1, looking for command line arguments that would match those of an encoded command. The parameter that we are looking for is "-encodedcommand" but the bare minimum that PowerShell needs provided as arguments to threat it as an encoded command is simply "-e". So, let's filter out for event id 1, look for all powershell processes (powershell.exe or PowerShell in description), and parameter that contains "-e" - final query is shown below:

```
(winlog.event_data.Description:*PowerShell OR  winlog.event_data.Image:*powershell.exe) AND
```

Once executed, we receive the following output:


1 hit
New Save Open Share Inspect
Filters (winlog.event_data.Description:*PowerShell OR win KQL Last 15 years Show dates Refresh
Jul 4, 2007 @ 10:44:53.292 - Jul 4, 2022 @ 10:44:53.292 — Auto
@timestamp per 30 days
Time _source

```
Feb 2, 2020 @ 22:57:33.107  winlog.event_data.Description: Windows PowerShell
winlog.event_data.CommandLine: "C:\WINDOWS\System32\WindowsPowerShell
\v1.0\powershell.exe" -ec dwBoAG8AYQBtAGkA winlog.event_data.Image: C:\Windows
\System32\WindowsPowerShell\v1.0\powershell.exe winlog.event_id: 1
```

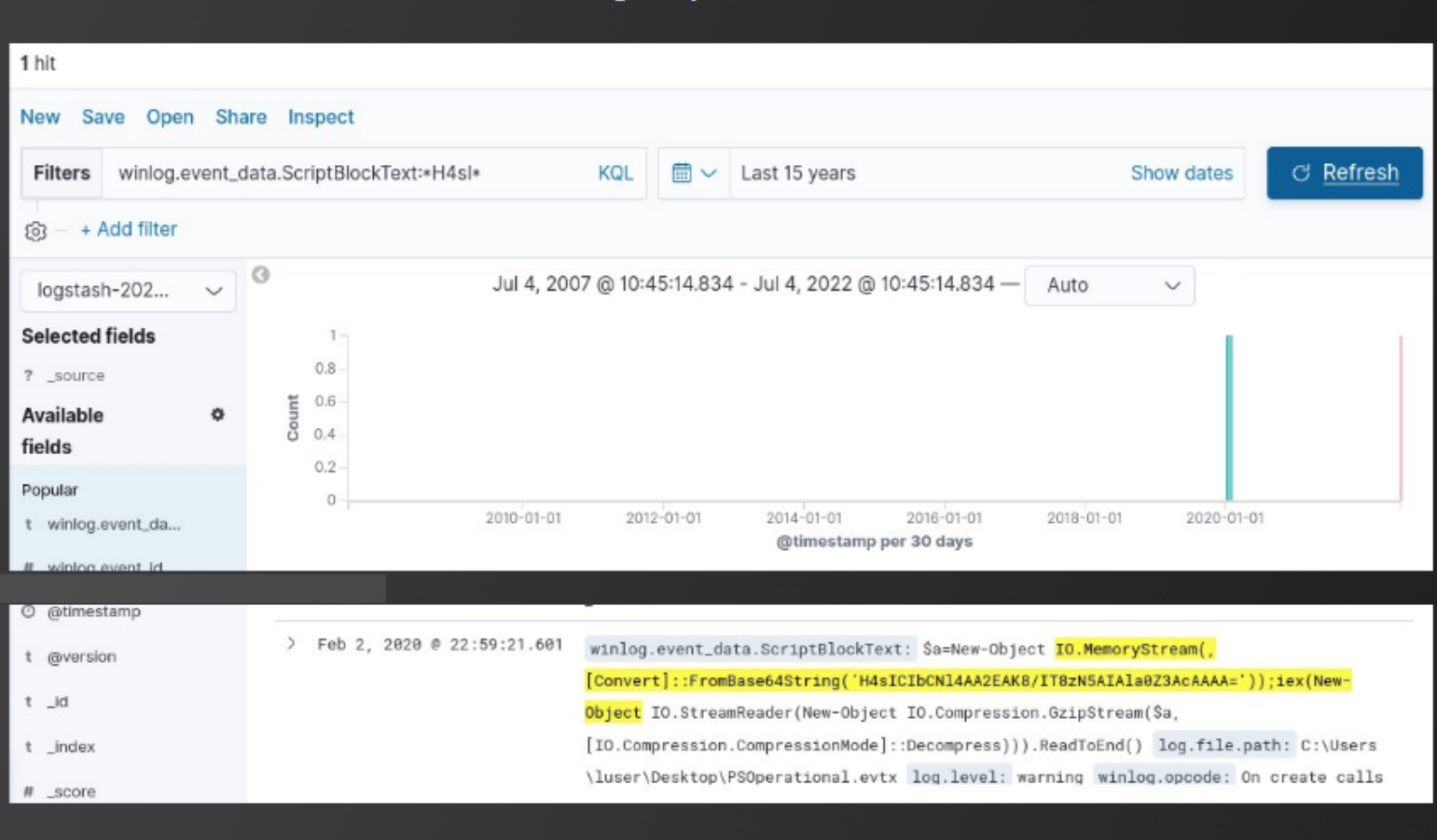
If we decode the passed command, we'll find that it is the command "whoami".

Task 5. Perform a hunt for PowerShell attacks utilizing GZIP compression

For this task, we'll be looking at ScriptBlockText of event id 4104. After some research we find that GZIP archives have the magic number "H4sI", which appears quite unique. That will be our detection -- let's filter for event id 4104 and final query below:

```
winlog.event_data.ScriptBlockText:*H4sI*
```

Once executed, we receive the following output:


1 hit
New Save Open Share Inspect
Filters winlog.event_data.ScriptBlockText:*H4sI* KQL Last 15 years Show dates Refresh
Jul 4, 2007 @ 10:45:14.834 - Jul 4, 2022 @ 10:45:14.834 — Auto
@timestamp per 30 days

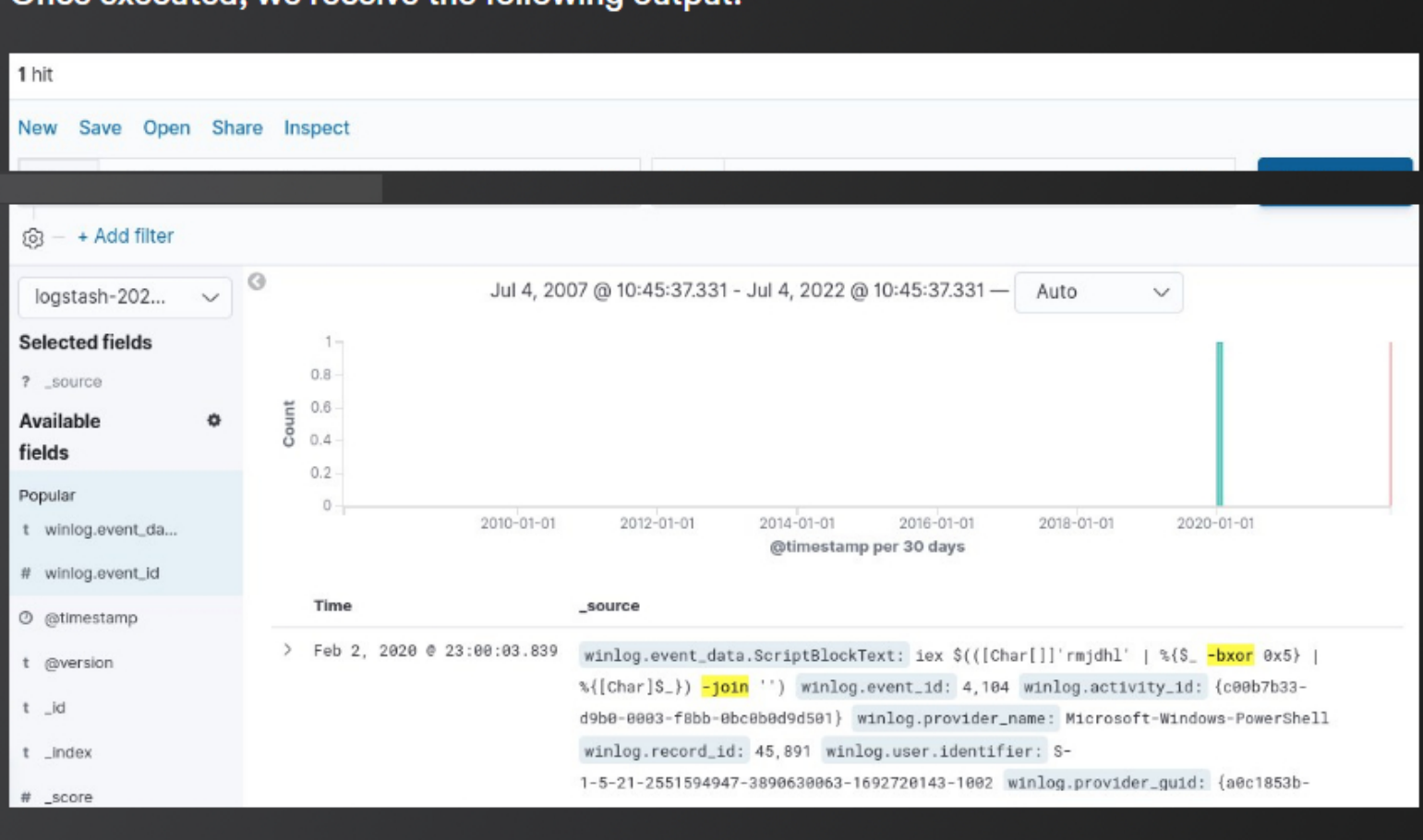
```
Feb 2, 2020 @ 22:59:21.601  winlog.event_data.ScriptBlockText: $a=New-Object IO.MemoryStream(,
[Convert]::FromBase64String("H4sI ...
Object IO.StreamReader(New-Object IO.Compression.GzipStream($a,
[IO.Compression.CompressionMode]::Decompress))).ReadToEnd() log.file.path: C:\Users
\user\Desktop\PSOperational.evtx log.level: warning winlog.opcode: On create calls
```

Task 6. Perform a hunt for obfuscated PowerShell code using XOR

For this task, we'll be looking at ScriptBlockText of event id 4104. After an extensive research, we find that XOR usage involves the operators "char", "bxor" and "join". After filtering for event id 4104, we construct a query searching only for the operators "bxor" and "join" as a more broad perspective to the search:

```
winlog.event_data.ScriptBlockText:(*bxor* AND *join*)
```

Once executed, we receive the following output:


1 hit
New Save Open Share Inspect
Jul 4, 2007 @ 10:45:37.331 - Jul 4, 2022 @ 10:45:37.331 — Auto
@timestamp per 30 days
Time _source

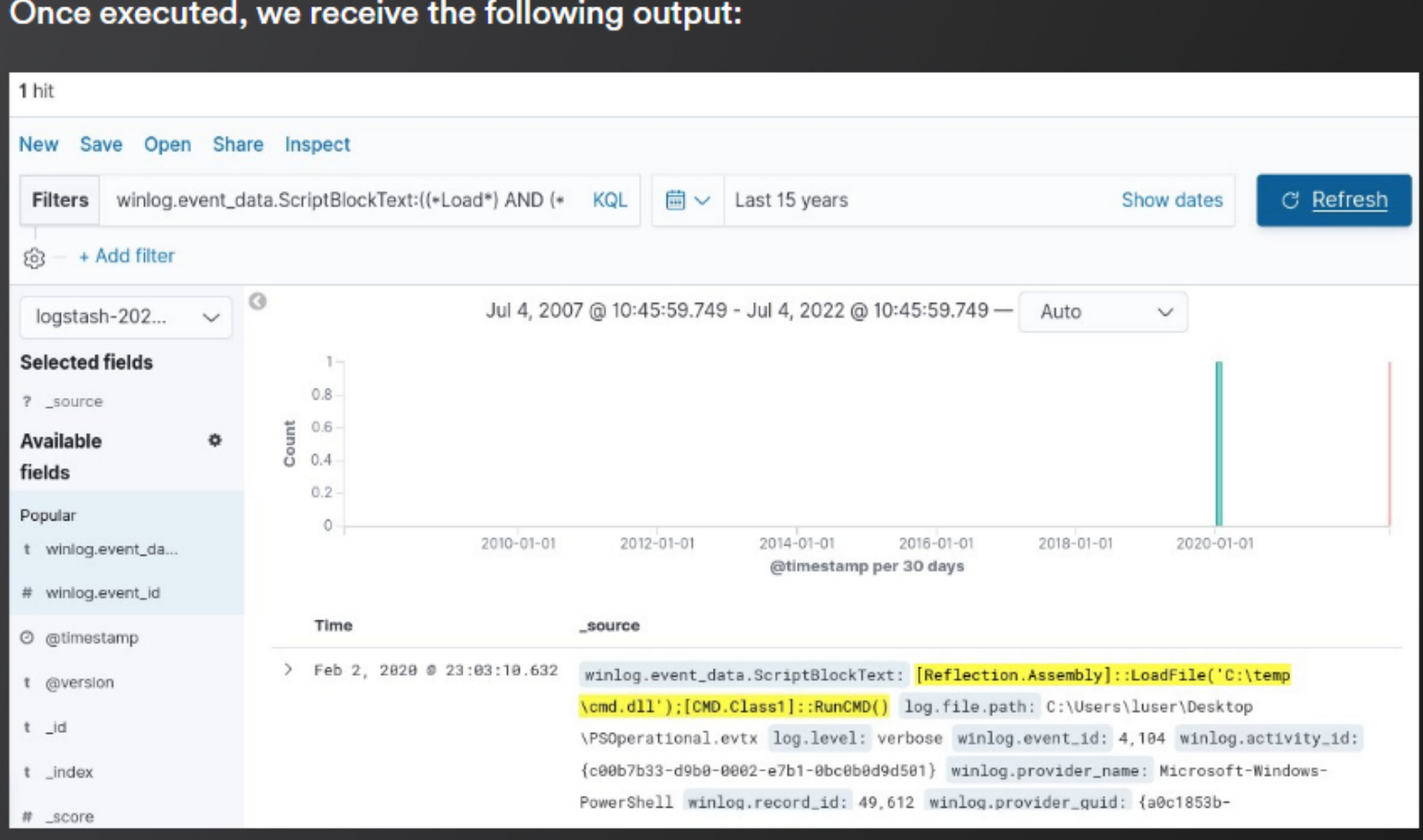
```
Feb 2, 2020 @ 23:00:03.839  winlog.event_data.ScriptBlockText: iex ${([Char[]]'rmjdhl' | %{$_ -bxor 0x5} |
%{[Char]$_}) -join ''} winlog.event_id: 4,104 winlog.activity_id: {c00b7b33-
d9b0-0003-f8bb-0bcd0b0d9d501} winlog.provider_name: Microsoft-Windows-PowerShell
winlog.record_id: 45,891 winlog.user.identifier: S-
1-5-21-2581594947-3890638063-1692728143-1002 winlog.provider_guid: {a0c1853b-
```

Task 7. Perform a hunt for execution of an assembly from file by PowerShell

For this task, we'll be looking at ScriptBlockText of event id 4104. After performing a research, we find that to execute an assembly from file, a function "Load" together with either "ReadAllBytes" or "LoadFile" is utilized. After filtering for event id 4104, we construct the final query as:

```
winlog.event_data.ScriptBlockText:((*Load*) AND (*ReadAllBytes* OR *LoadFile*))
```

Once executed, we receive the following output:


1 hit
New Save Open Share Inspect
Filters winlog.event_data.ScriptBlockText:((*Load*) AND (* KQL Last 15 years Show dates Refresh
Jul 4, 2007 @ 10:45:59.749 - Jul 4, 2022 @ 10:45:59.749 — Auto
@timestamp per 30 days
Time _source

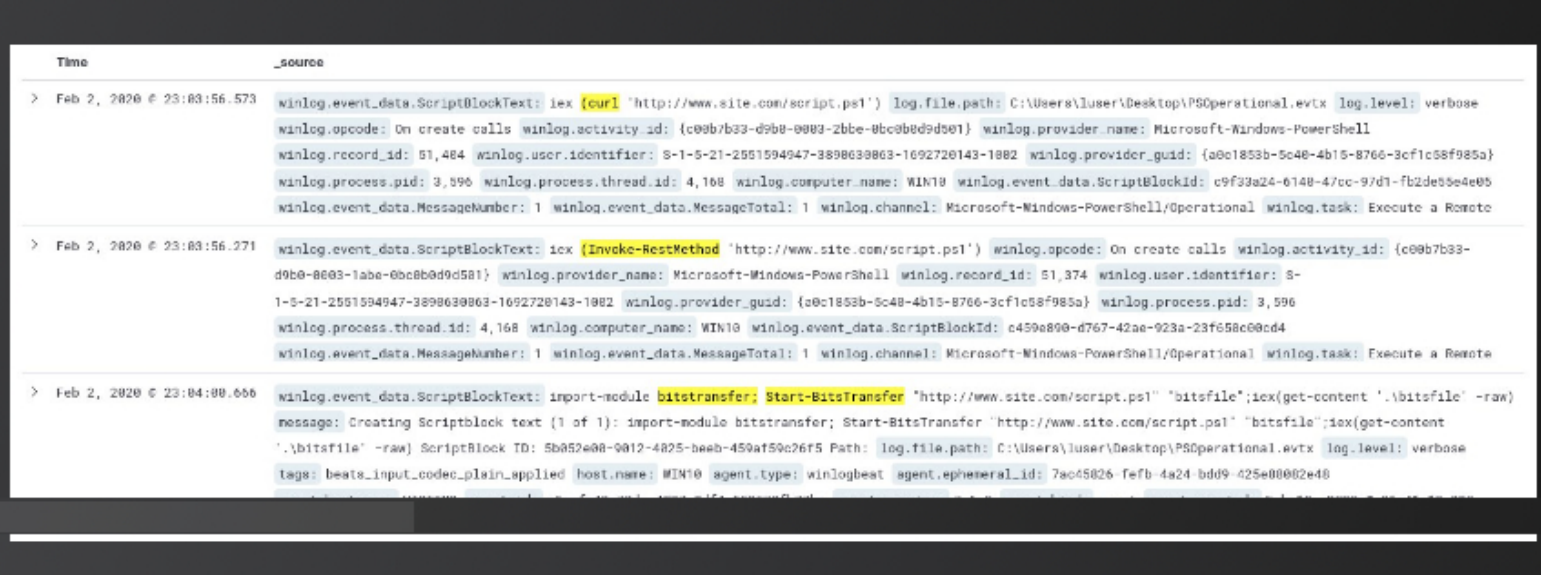
```
Feb 2, 2020 @ 23:03:10.632  winlog.event_data.ScriptBlockText: [Reflection.Assembly]::LoadFile('C:\temp
\cmd.dll');[CMD.Class]::RunCMD() log.file.path: C:\Users\user\Desktop
\PSOperational.evtx log.level: verbose winlog.event_id: 4,104 winlog.activity_id:
{c00b7b33-d9b0-0002-e7b1-0bcd0b0d9d501} winlog.provider_name: Microsoft-Windows-
PowerShell winlog.record_id: 49,612 winlog.provider_guid: {a0c1853b-
```

Task 8. Perform a hunt for PowerShell commands downloading content

For this task, we'll be looking at ScriptBlockText of event id 4104. After doing our research, we identify a large number of possibilities for download content. The final query with all of them included is shown below:

```
winlog.event_data.ScriptBlockText:(*WebClient* OR *DownloadData* OR *DownloadFile* OR *Downlo
```

Once executed, we receive the following output, which identifies 3 distinctive log entries:



The search detected download through the commands:

- Start-BitsTransfer
- Curl
- Invoke-RestMethod

At this point, you should also be aware that COM objects can be used to perform file download, therefore, the following query should also be incorporate in the search. In our research, we identified the following CLSIDs to be interesting:

```
"0002DF01-0000-0000-C000-000000000046" | InternetExplorer.Application
"F6D90F16-9C73-11D3-B32E-00C04F990BB4" | Msxml2.XMLHTTP
"F5078F35-C551-11D3-89B9-0000F81FE221" | Msxml2.XMLHTTP.3.0
"88D969C6-F192-11D4-A65F-0040963251E5" | Msxml2.XMLHTTP.6.0
"AFBA6B42-5692-48EA-8141-DC517DCF0EF1" | Msxml2.ServerXmlHttp
"AFB40FFD-B609-40A3-9828-F88BBE11E4E3" | Msxml2.ServerXmlHttp.3.0
"88D96A0B-F192-11D4-A65F-0040963251E5" | Msxml2.ServerXmlHttp.6.0
"2087C2F4-2CEF-4953-A8AB-66779B670495" | WinHttp.WinHttpRequest.5.1
```

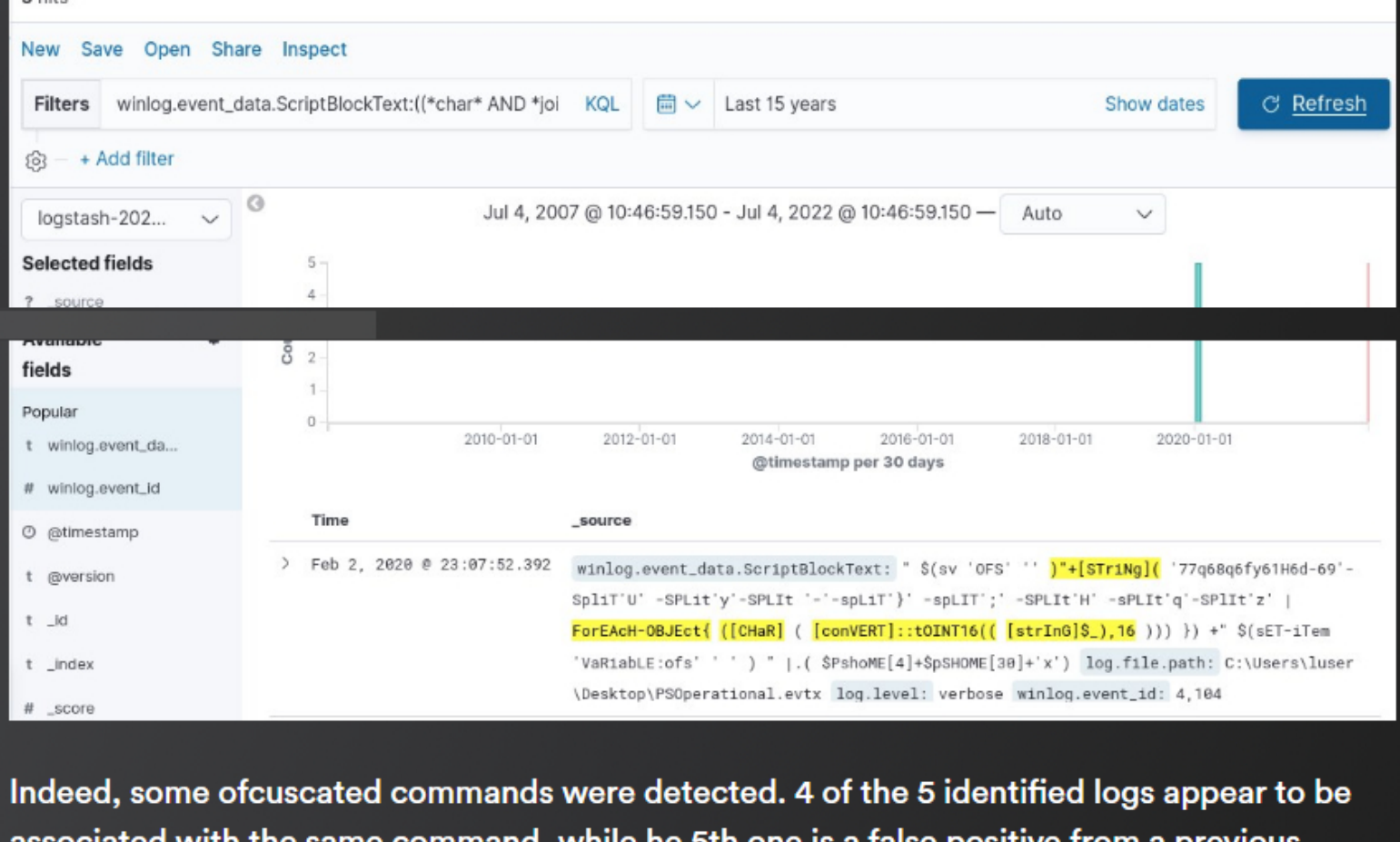
```
"00023500-0000-0000-C000-000000000046" | Excel.Application
```

Task 9. Perform a hunt for obfuscated PowerShell commands

For this task, we'll be looking at ScriptBlockText of event id 4104. After doing a research on obfuscated commands, we identify a large number of interesting characters that may be of interest, which compose the final query as:

```
winlog.event_data.ScriptBlockText:((*char* AND *join*) OR ((*ToInt* OR *ToInt16* OR *ToDecima
```

Once executed, we receive the following output:


5 hits
New Save Open Share Inspect
Filters winlog.event_data.ScriptBlockText:((*char* AND *joi KQL Last 15 years Show dates Refresh
Jul 4, 2007 @ 10:46:59.150 - Jul 4, 2022 @ 10:46:59.150 — Auto
@timestamp per 30 days
Time _source

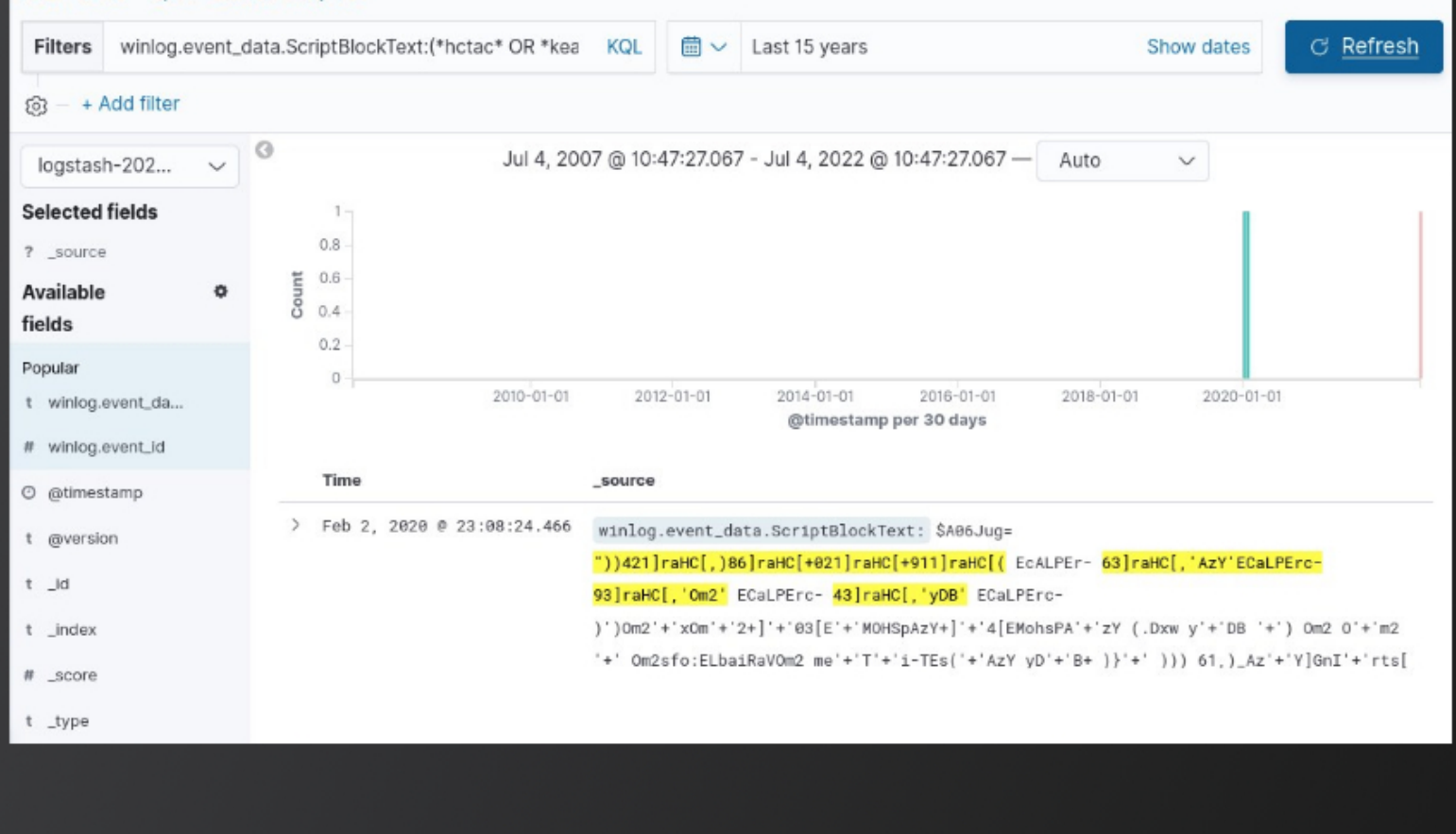
```
Feb 2, 2020 @ 23:07:52.392  winlog.event_data.ScriptBlockText: " $(sv 'OFS' '' )"+[STrINg]( '77g68q6fg61h6d-69'-
SplIt'G' -SPLit'y'-SPLIt '-'-spLIt']' -SPLIt'H' -sPLit'g'-SPlIt'e' |
ForEAcH-OBJecT{ ([CHaR] ( [cOnVERT]::tOINT16( [strInG]$_),16 ))}) )+ " $(sET-iTem
'VaRiaBLE:oFs' ' ' ) ".( $PshoME[4]+$pshOME[30]+'x') log.file.path: C:\Users\user
\Desktop\PSOperational.evtx log.level: verbose winlog.event_id: 4,104
```

Indeed, some ofcuscated commands were detected. 4 of the 5 identified logs appear to be associated with the same command, while he 5th one is a false positive from a previous exercise. Therefore, we have identified only 1 command so far, we should investigate for more obfuscation techniques to find others.

Additional research leads us to reverse obfuscated, or certain words spelled backwards. We go through common cmdlets, and terms and construct a query with them in reverse order as follows:

To detect the second obfuscated command, we utilize the following query, which identifies PowerShell commands spelled backwards:

Once executed, we receive the following output:


1 hit
New Save Open Share Inspect
Filters winlog.event_data.ScriptBlockText:(*ctac* OR *kes KQL Last 15 years Show dates Refresh
Jul 4, 2007 @ 10:47:27.067 - Jul 4, 2022 @ 10:47:27.067 — Auto
@timestamp per 30 days
Time _source

```
Feb 2, 2020 @ 23:08:24.466  winlog.event_data.ScriptBlockText: IX0Lkug+
')421]raHC[,)06]raHC[+021]raHC[+911]raHC[( EcALPEr- 63]raHC[,'AzY'EcALPEr-
63]raHC[,'Om2' EcALPEr- 43]raHC[,'yOK' EcALPEr-
```

KEY TAKEAWAYS

1. PowerShell has a number of techniques that can be utilized to circumvent detection and analysts.